

Contents

Abstract	2
1 Introduction	2
2 Inception phase	3
2.1 Specification (Recap)	3
3 Elaboration phase	3
3.1 Requirements elicitation (using FURPS+)	3
3.2 Use case	4
3.3 Domain model	5
3.4 Development time for this project	6
3.5 Architecture for the application(Client-Server Tiers)	7
3.6 List of requirements according to ChatGPT	7
3.7 Domain model according to ChatGPT	8
3.8 Observations	8
3.9 First version of the class diagram, based on the façade pattern	8
3.10 First version prototype	9
4 Construction phase	11
4.1 Use case Analyses	11
4.2 FDUCD	12
4.3 System Sequence Diagram	13
4.4 Interaction diagram	13
4.5 Evaluation of the domain model for super and sub classes	15
4.6 Class diagram	15
4.7 Object diagram	17
4.8 Implementation	18
4.9 Testing	18
5 Discussion	18
6 Summary	18
Acknowledgments	18
References	18

Software development based on user specification

GODSPOWER OGAGA UTI*, University of South-Eastern Norway, Norway

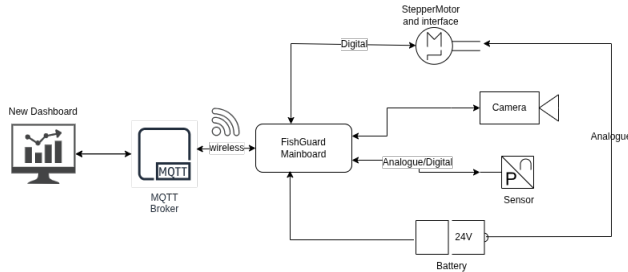


Fig. 1. FISHGUARD AI, 2026.

FISHGUARD AI is an imaginary company that produces buoyancy vehicles used for underwater data acquisition in fish farms and is based in Stavanger, Norway.

CCS Concepts: • **Software and its engineering** → **Requirements analysis; Software design engineering; Software design techniques; Object oriented development; Software prototyping.**

Additional Key Words and Phrases: Software, OOAD, OOP, FURP+, OOSAD, UP, UML

Godspower Ogaga Uti. 2026. Software development based on user specification. *J. ACM* 37, 4, Article 111 (April 2026), 18 pages. <https://orcid.org/my-orcid?orcid=0009-0006-1817-7789>

1 Introduction

Today, fish farming in Norway is prolific along most of the country's coastline, producing more than 1,5 million tonnes of fish per year, almost all of which is exported. In 2017, Norway accounted for over 52% of the world's Atlantic salmon production. Mortality levels are high in Norwegian fish farms. In 2023, 37.7 million farmed salmon smolts died, along with 62.8 million salmon during the seawater phase. Altogether, 100.5 million farmed salmon died that year. The major causes of fish death include gill disease, winter ulcers, and intense handling, such as during delousing procedures[Dyrevern 2024]. As part of a proactive measure, FISHGUARD AI developed a buoyancy vehicle for underwater data acquisition. The device has a camera and an embedded IoT system running an edge image recognition machine learning algorithm to detect sick fish in a fish farm, helping farmers with early intervention. The current system also has sensors that measure temperature, pressure, and other aquatic conditions of the farm. The device uses a TF(TransFlash) memory card to store collected data. At the end of each dive, farmers can extract data from the device. The device uses Wi-Fi and Bluetooth for external connectivity. However, many farmers struggle to make meaningful decisions from the overall data collected.

Author's Contact Information: Godspower Ogaga Uti, gouti6308@usn.no, University of South-Eastern Norway, Porsgrunn, Norway.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-735X/2026/4-ART111

<https://orcid.org/my-orcid?orcid=0009-0006-1817-7789>

2 Inception phase

2.1 Specification (Recap)

Due to growing market demand for this device and client requests, the directors of FISHGUARD AI have decided to develop an optional software dashboard for their clients. The company maintains a list of its clients along with their respective issued device numbers and has a website, which they intend to extend to accommodate the new information system. The software dashboard will be subscription-based and will provide the following functionalities; users can login or create an account and subscribe based on their device number, the system collect and store data from the underwater vehicle, remote access and control of the underwater vehicle, provides predictive analytics for risk assessment and an automated recommendations based on data history.

3 Elaboration phase

3.1 Requirements elicitation (using FURPS+)

- *Functionality*
- *Usability*
- *Reliability*
- *Performance*
- *Supportability*
- + indicates the additional subcategories
 - *Implementation requirements*
 - *Interface requirements*
 - *Operations requirements*
 - *Packaging requirements*
 - *Legal requirements*

3.1.1 Functional requirements.

- The system displays a login page for users to login or create account using their details and device number.
- The system shall accept orders over the Internet.
- The system shall provide a secure means of allowing clients to pay by credit card.
- The system collects data from the buoyancy vehicle and saves it.
- The system has multiple pages displaying risk assessments based on historical data and recommendations respectively.
- The system allows the user to controls the buoyancy vehicle (Switching between different modes of operation).

3.1.2 Nonfunctional requirements.

Quality requirements (URPS).

- Any user who knows how to read graphs and understands flow charts should be able to use the system without the user manual. [Usability requirement]
- The system must ensure that the data displayed is up to date.[Performance requirement]
- None [Supportability requirement][Bruegge 2014].

+ Requirements (Constraints or Pseudo requirements for the dashboard software).

- The software for the dashboard will be written using C# and Vue.js(Web page), to comply with current company policy.[Implementation requirement]

- The dashboard system will use MongoDB to store sensor data, which is the company's current database management system.[Implementation requirement]
- The communication protocols and interfaces must comply with MQTT Protocol.[Interface requirement]
- What if the MQTT broker stopped working during operation?[Operations requirements]

3.2 Use case

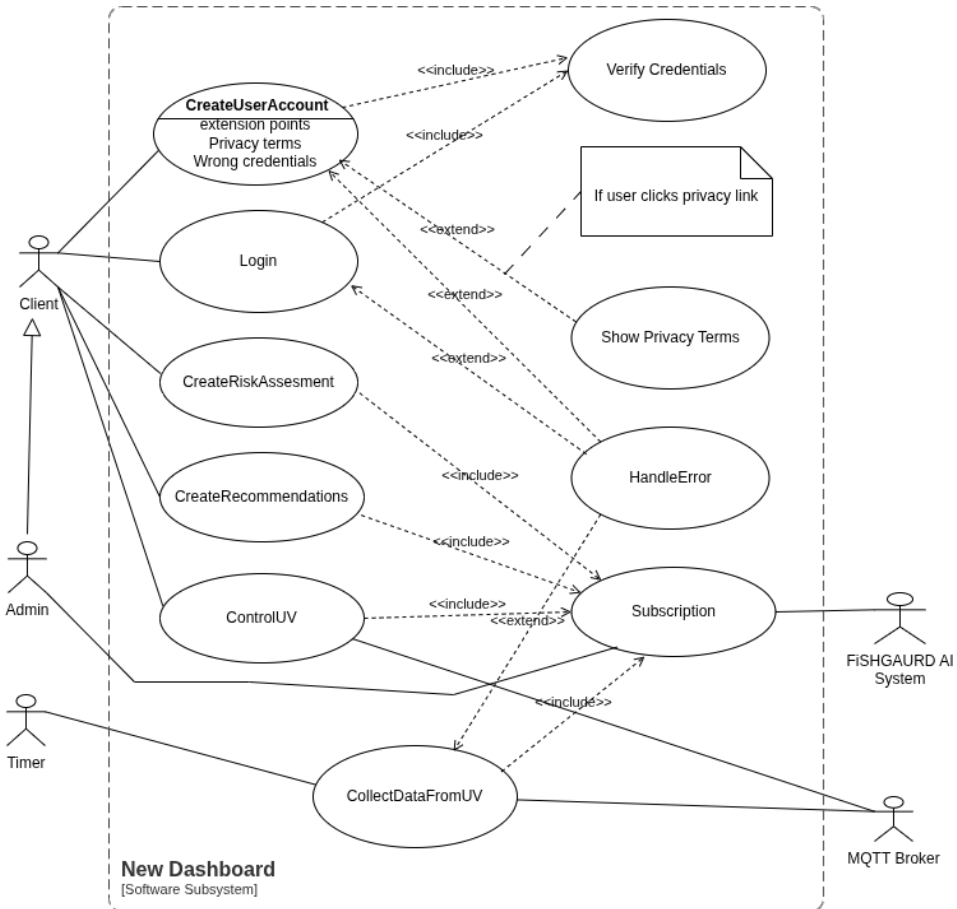


Fig. 2. Use case diagram for the dashboard.

3.3 Domain model

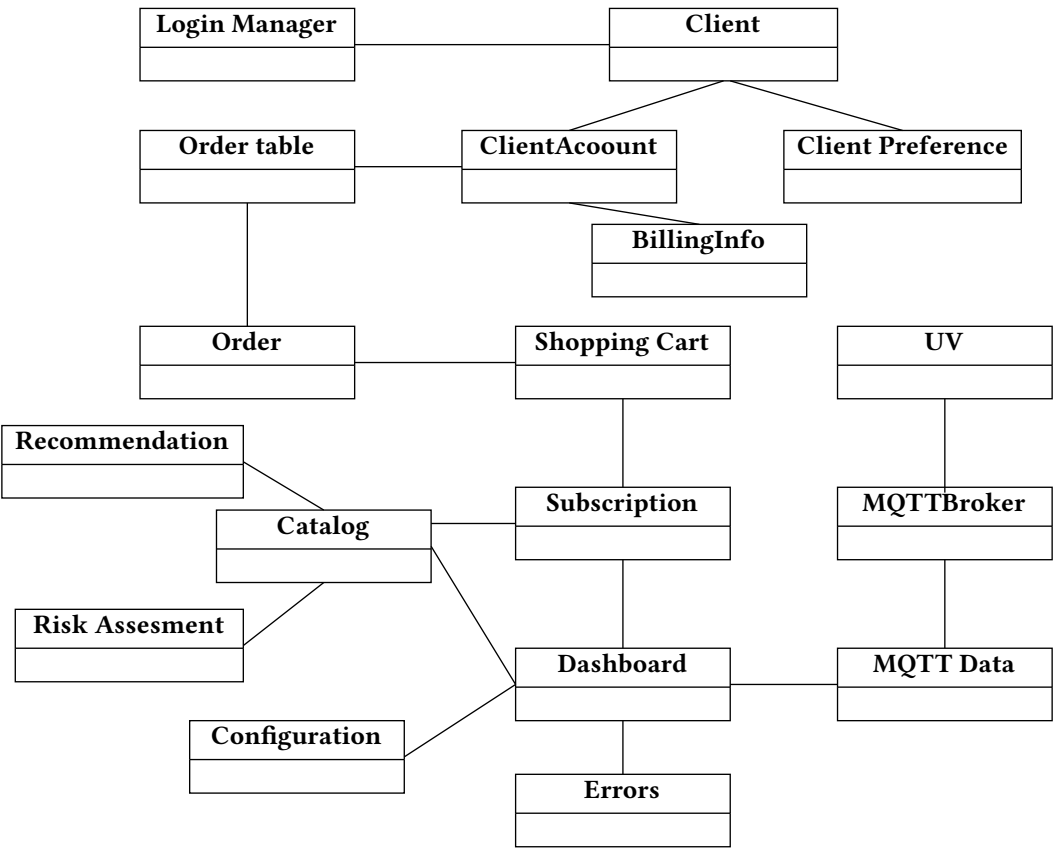


Fig. 3. Domain model for the dashboard.

3.4 Development time for this project

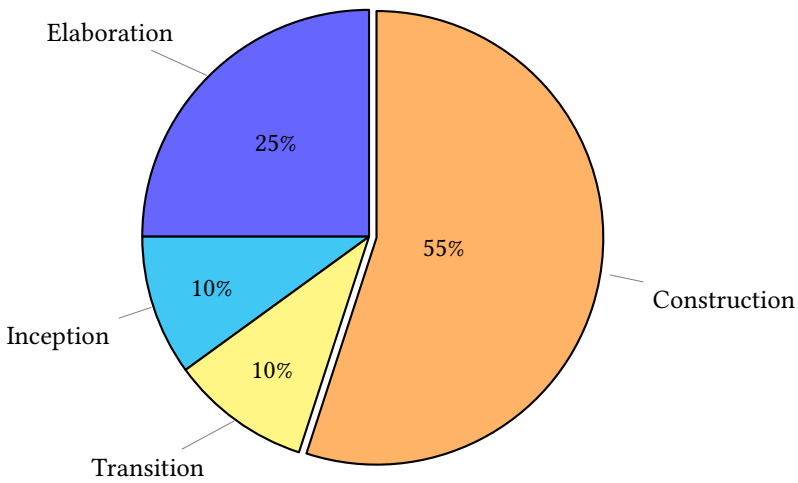


Fig. 4. Time distribution of the unified process phases
[Braude and Bernstein 2016a]

- Each iteration will take 3 weeks.
- Total number of iterations = #1 iteration (Elaboration phase) + # number iterations=number of uses cases(Construction) + 1 iteration(Transition)
- Development time = Total number of iterations x length of each iteration = 36 weeks.

3.5 Architecture for the application(Client-Server Tiers)

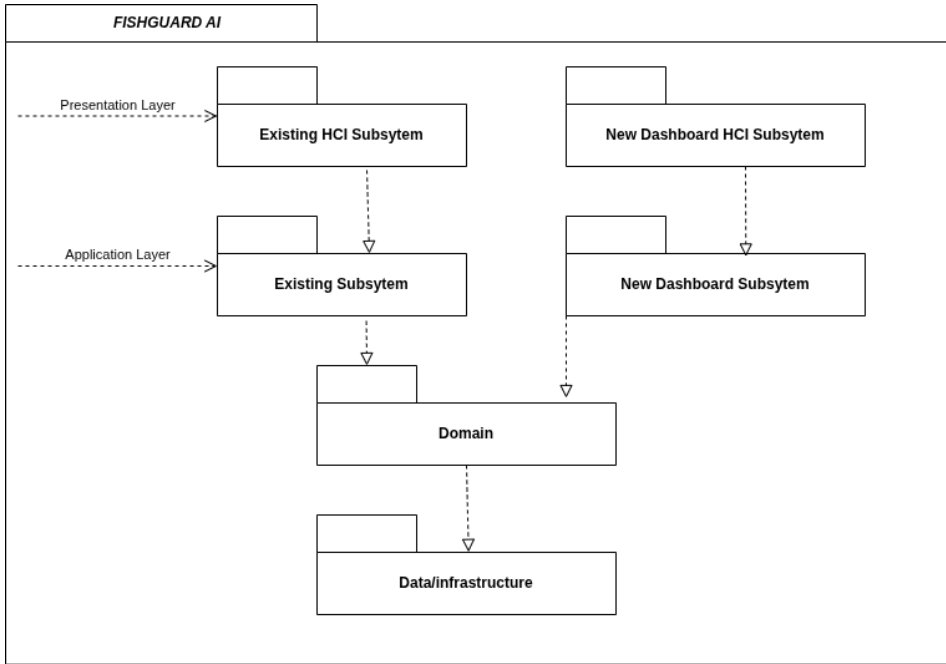


Fig. 5. Four layer architecture (Partitioned subsystems).
[Bennett and Farmer 2010]

Figure 5 shows a class diagram of the selected layered architecture of the application. The Data/Infrastructure layer provides access to a database that contains all the details of the clients, their device and their sensor data. The Domain layer provides common functionality for the application layer subsystems. Finally, each application layer has its presentation layer, thus providing modularity and high levels of encapsulation.

3.6 List of requirements according to ChatGPT

3.6.1 Functional requirements.

- User registration and login
- Device linking via device number
- Data upload and storage
- Remote device control
- Predictive analytics
- Automated recommendations

3.6.2 Nonfunctional requirements.

- Security (data encryption, authentication)
- Scalability (growing number of farms/devices)
- Reliability (stable IoT communication)
- Performance (real-time insights)

3.7 Domain model according to ChatGPT

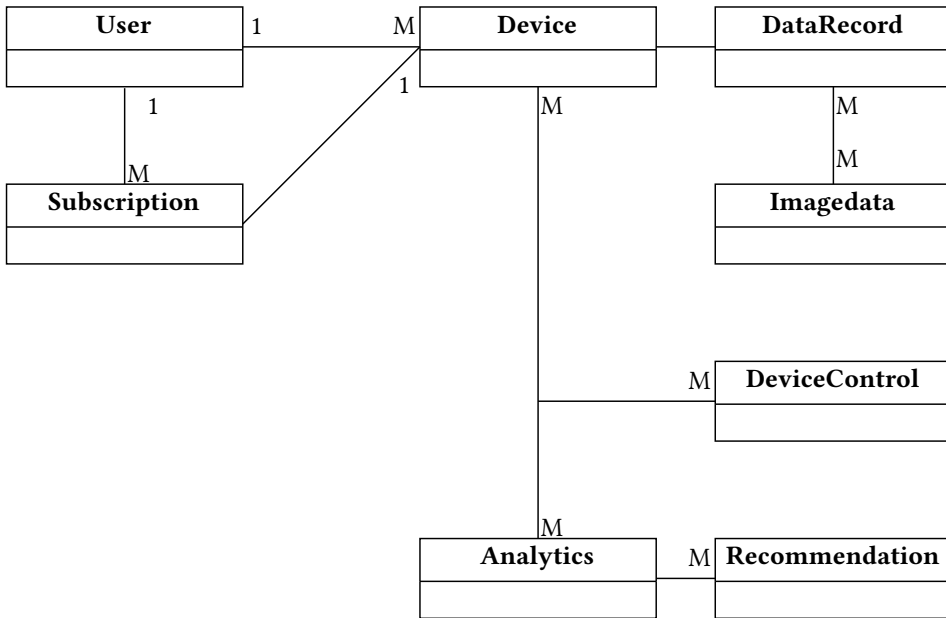


Fig. 6. Domain model for the dashboard (ChatGPT).

3.8 Observations

The results from ChatGPT for the list of requirements are similar to mine, perhaps complimentary. While there is a little difference in the domain models. The domain model from ChatGPT didn't capture much, and every association has an explicit multiplicity. However, dealing with multiplicity in domain modelling should be avoided, as it is time-consuming and can impair analysis [Rosenberg and Scott 2001]. Furthermore, It is better to overspecify a domain model with lots of fine-grained conceptual classes than to underspecify it [Larman 2001]. While ChatGPT is a helpful tool, one cannot rely on it to trust all its answers. Their results are usually based on prediction.

3.9 First version of the class diagram, based on the façade pattern

Façade Pattern summary. - Define a Singleton that is the sole means for obtaining functionality from the package. A Singleton ensures that there is exactly one instance of a class. And its instance can be obtained from anywhere in the application [Braude and Bernstein 2016b].

The control classes can be mapped into the Application Logic Layer and entity classes into domain.

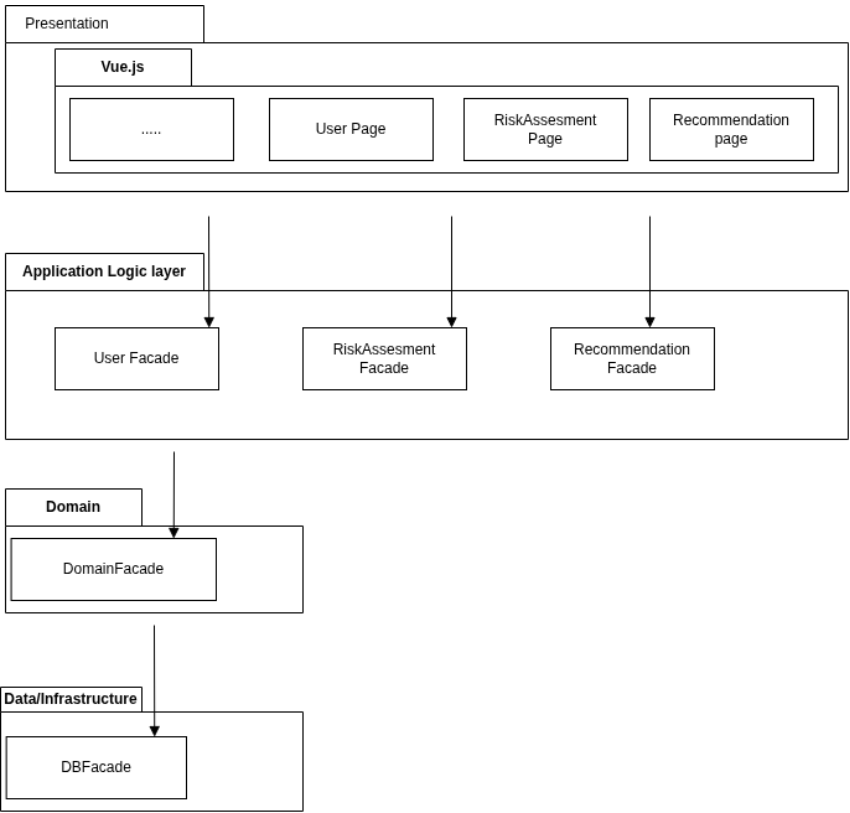


Fig. 7. Class diagram showing the selected layered architecture of the application

3.10 First version prototype

3.10.1 Human-computer interaction (HCI). Figure 8, 9 and 10 shows the user web interface prototype implemented using Vue.js.

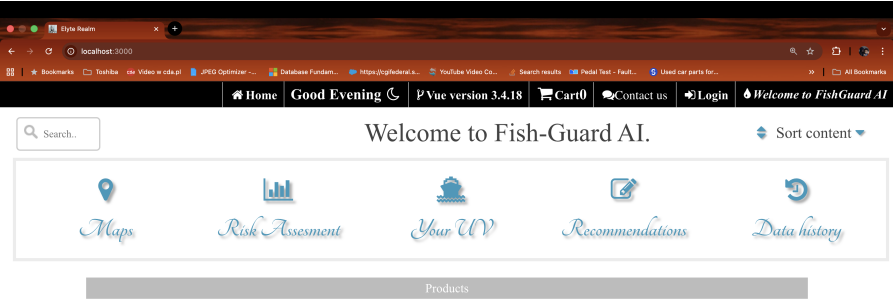


Fig. 8. New dashboard HCI subsystem (Home page).

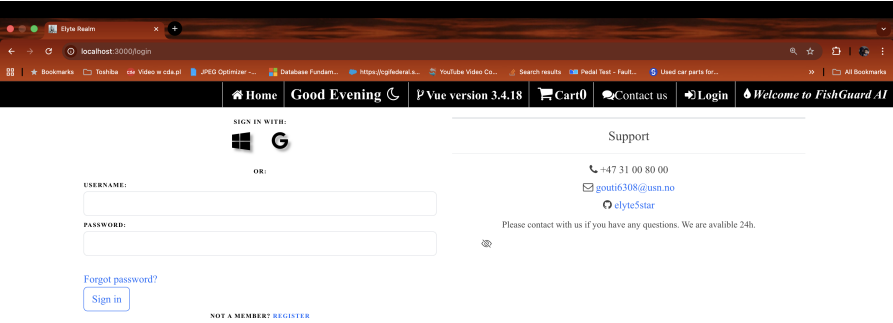


Fig. 9. New dashboard HCI subsystem (Login page).

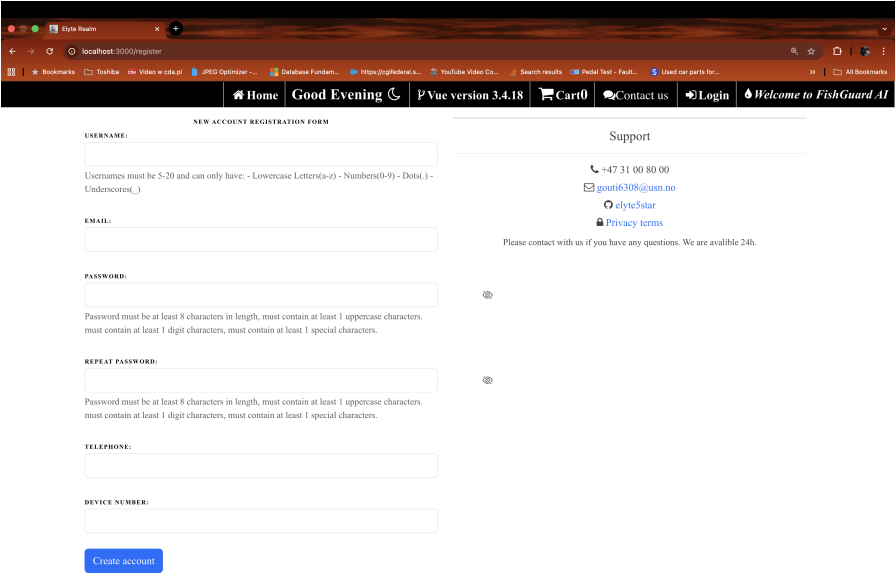


Fig. 10. New dashboard HCI subsystem (Create User page).

4 Construction phase

4.1 Use case Analyses

The “Collect Data from UV” use case is considered the most important because it serves as the core functionality of the new dashboard system. The Analytics part of the dashboard depends on the data from the UV.

4.2 FDUCD

Table 1. Fully dressed use case document (FDUCD)

Use case name	Collect Data from UV.
Scope	New Dashboard
Level	User goal
Participating actors	Timer, MQTT Broker
Precondition	The User/Client has a valid subscription and UV is publishing data to broker
Success Guarantee	Data saved in Database.
Flow of events(Main Success Scenario)	(1) Connect the application to the MQTT Broker (2) Subscribe to MQTT sensors topics (3) Wait for messages from broker (4) Save the messages to database (5) Check for finished signal from broker.
Exit condition	• The dashboard displays acknowledgment at the end of each UV dive and shows the timestamp.
Extensions	• 1(a) If connection timeout error, send error to dashboard. • 1(b) If the user has no valid subscription, dont connect to MQTT broker. Send error to dashboard. • 3(a) If no incoming messages for 10mins, goto 5

4.3 System Sequence Diagram

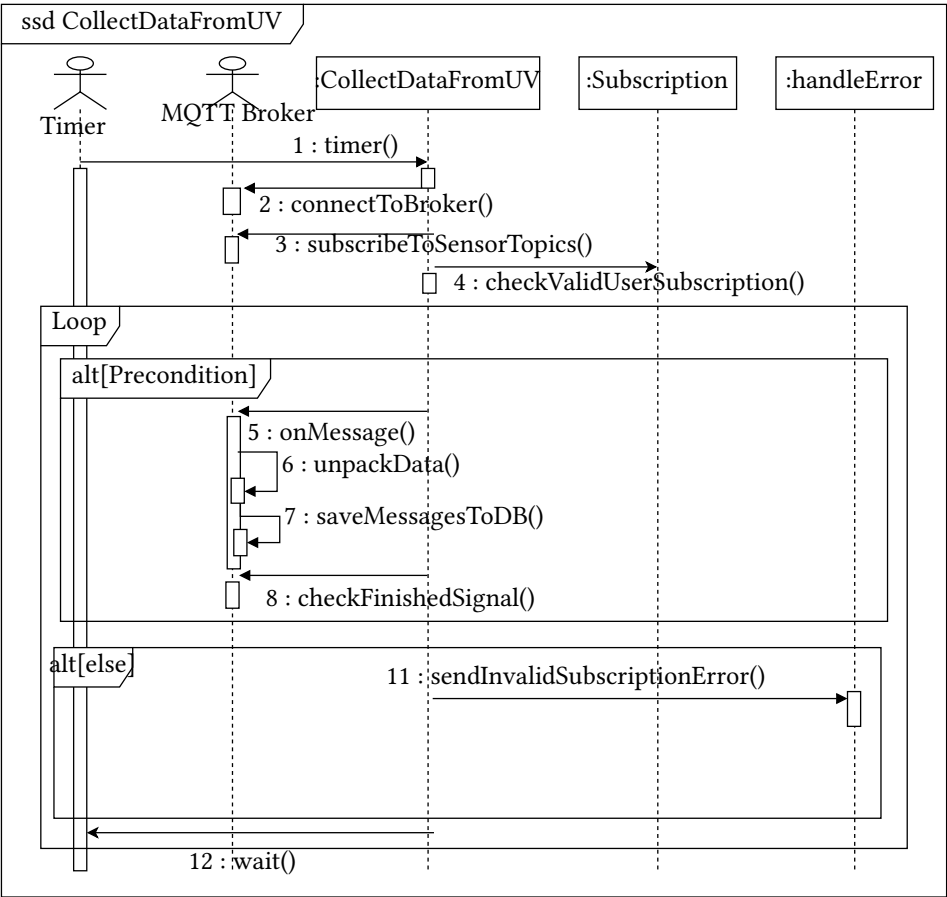


Fig. 11. System Sequence diagram for the CollectDataFromUV use case.[Bruegge 2014]

Design.

4.4 Interaction diagram

The interaction diagram for the CollectDataFromUV use case will be a Collaboration (Communication) diagram, since Information (data) is the primary focus, not time[Skeie 2023]. Messages sent and received by an object is more visible using this technique.

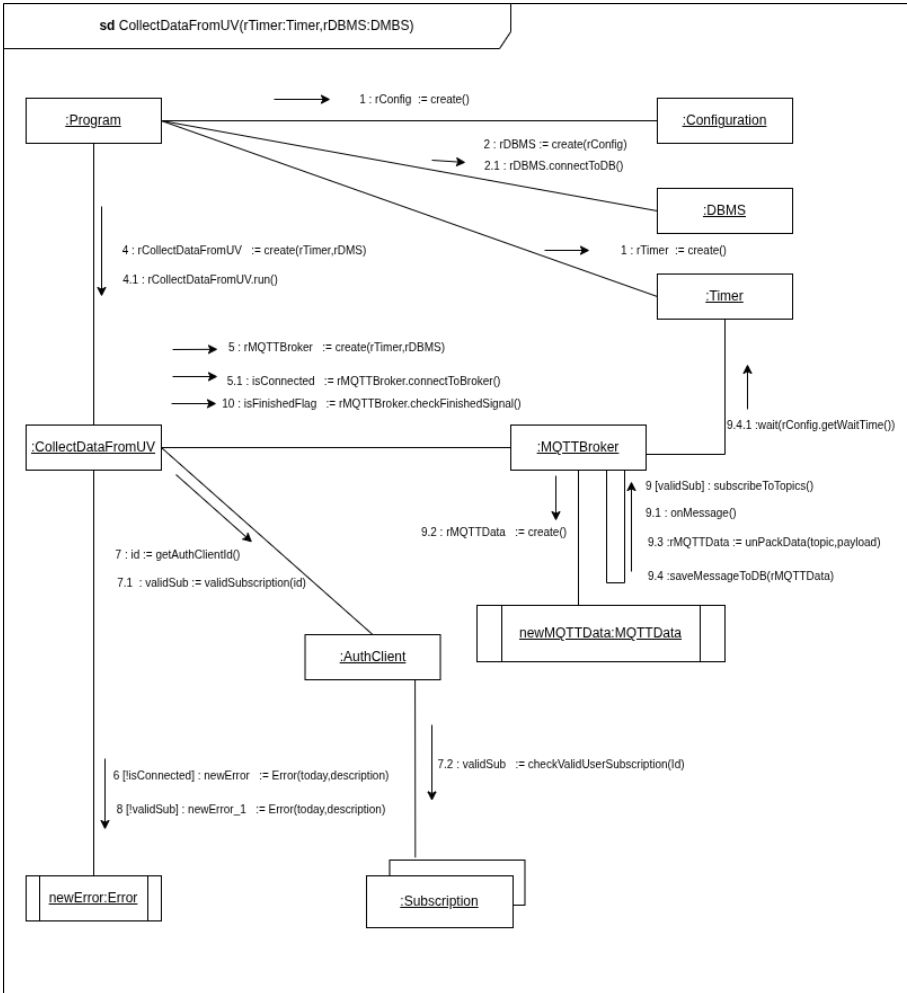


Fig. 12. Communication diagram for the use case CollectDataFromUV

4.5 Evaluation of the domain model for super and sub classes

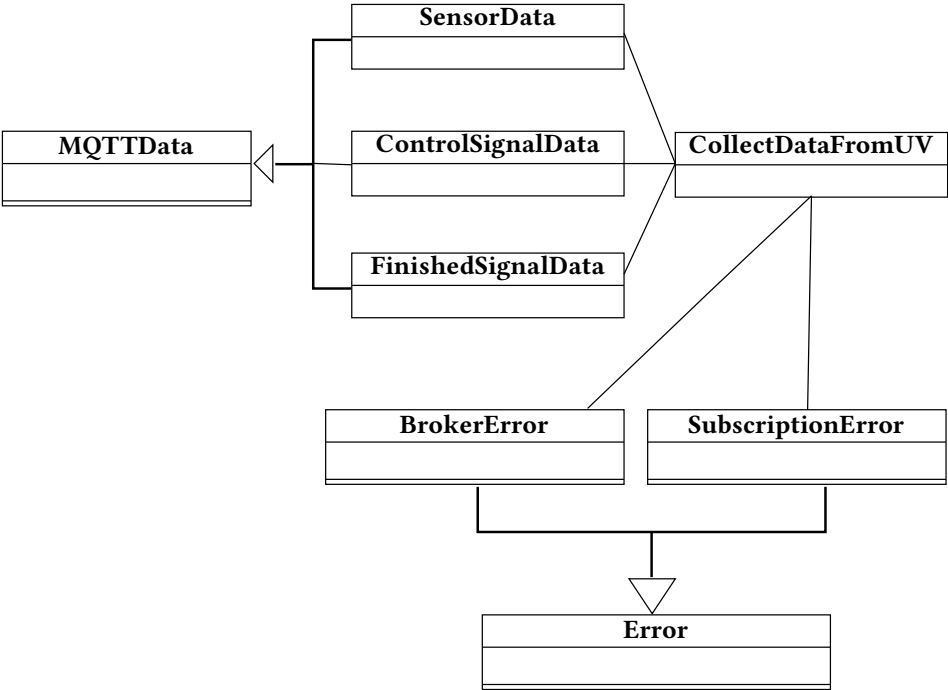


Fig. 13. Domain model for super and sub classes.

4.6 Class diagram

Patterns. Patterns of assigning responsibilities using GRASP.

Pattern Name	Class
Controller	CollectDatafromUV
Information expert	MQTTBroker
High cohesion	Each class has moderate responsibilities in one functional area, the methods in the classes solves a single specific problem .
Low coupling	High cohesion often mean low coupling
Creator	CollectDatafromUV, MQTTBroker and the final façade class

Table 2. Design patterns.

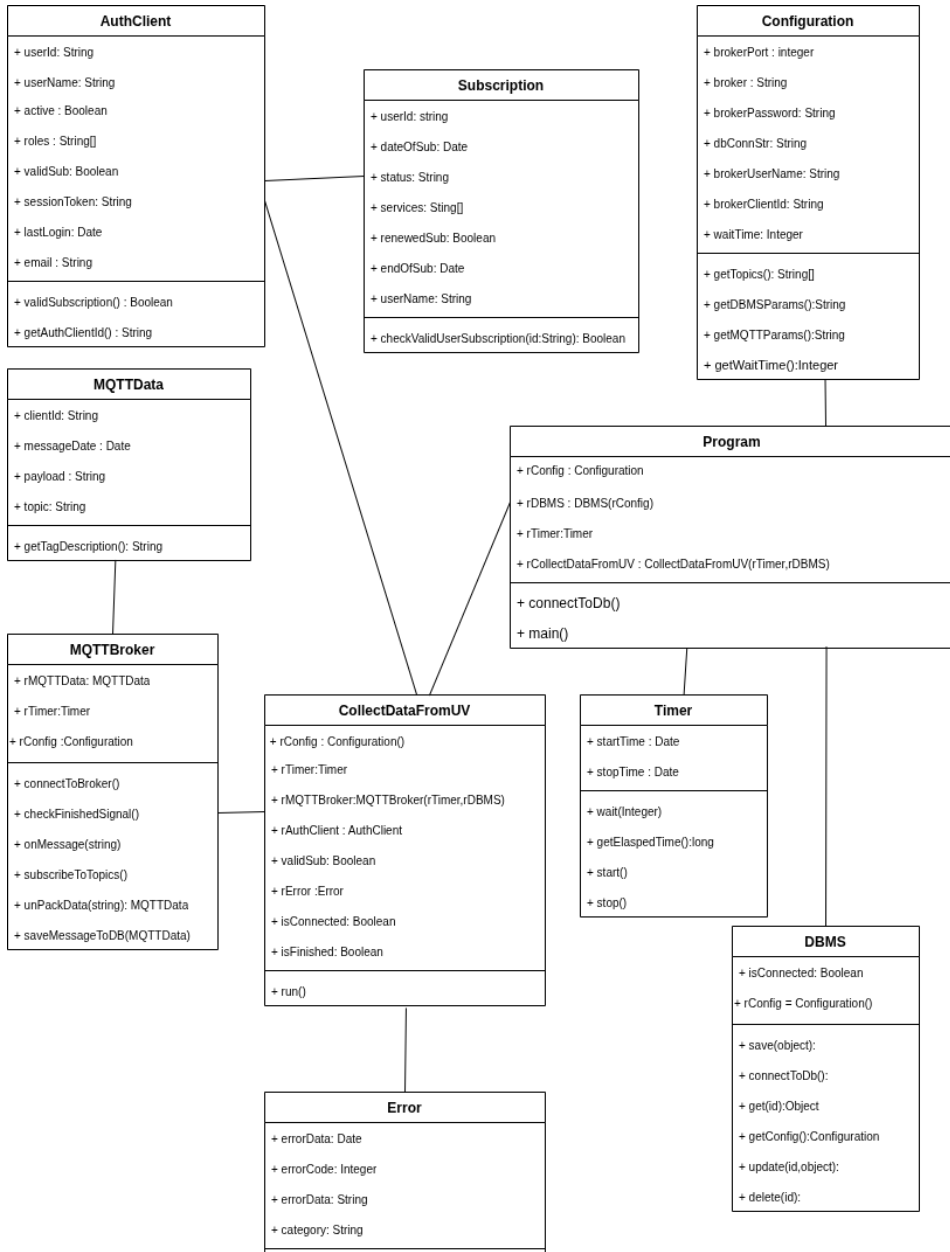


Fig. 14. A class diagram based on the interaction diagram for CollectDatafromUV use case.

4.8 Implementation

The Focus will be on the last three layers of the architecture, since the web UI will be implemented in a different language independently. The use case spawns from the Application logic layer to the Data(Infrastructure) layer.

4.9 Testing

Testing is a vital phase in software development because it is the only way to verify if the code implementation of a functionality meets its requirements. Events number 1-5 in table 1 and their extensions will be used to create test cases for the requirements.

5 Discussion

The web server was implemented using the ASP.NET Core 10 framework in C#. The UV will publish sensor data to the MQTT Broker, while the web server will subscribe to the sensor topics and act as a consumer. Node-RED was used to simulate the UV; it generates dummy sensor data and then publishes it to the broker. The data from the broker are stored in MongoDB, which is the company's existing database system. For development purpose, the MongoDB server, MQTT broker, and Node-RED were running in a Docker container.

6 Summary

This exercise provided valuable hands-on experience with software development using the Unified Software Development Process(USDP). The code is available on **GitHub**. The front-end prototype was implemented using Vue.js 3, a JavaScript framework. The interaction diagrams serve as the documentation of the software implementation.

Acknowledgments

To Nils-Olav Skei, for the mentorship.

References

- Simon Bennett and Ray Farmer. 2010. *Object-oriented Systems Analysis and Design* (4 ed.). McGraw-Hill International (U.K.) Limited, Chapter 7.1, 346–347.
- Eric J. Braude and Michael E. Bernstein. 2016a. *SOFTWARE ENGINEERING - Modern Approaches* (2 ed.). Waveland Press, Inc, Chapter 3.2.5.
- Eric J. Braude and Michael E. Bernstein. 2016b. *SOFTWARE ENGINEERING - Modern Approaches* (2 ed.). Waveland Press, Inc, Chapter 17.5.
- Bernd Bruegge. 2014. *Object-Oriented Software Engineering Using UML, Patterns, and Java*. (3 ed.). Pearson Education Limited, Essex, England, Chapter 1.4, 58.
- Dyrevern. 2024. *Fish farming in Norway*. Retrieved April 19, 2026 from <https://dyrevern.no/dyrevern/fish-farming-in-norway/>
- Craig Larman. 2001. *Applying UML and patterns- An introduction to Object-Oriented Analysis and Design and the Unified process* (2 ed.). Prentice hall, Upper Saddle River, NJ 07458, USA, Chapter 10.2.
- Doug Rosenberg and Kendall Scott. 2001. *Applying Use Case Driven Object Modeling with UML*. Pearson Education Corporate Sales Division, NJ, USA, Chapter 2, 180.
- Nils-Olav Skeie. 2023. *Object-oriented analysis, design, and programming using UML and C#*. Pearson Education Limited, Porsgrunn, Norway, Chapter 8.3, 339.

Received 28 April 2026; revised 28 April 2026